

Week 16: The Classic Machine-Learning Toolkit

Linear Decision Boundaries, Trees, Random Forests, and Honest Evaluation

Instructor / Mentor

Kiki Research Training Program

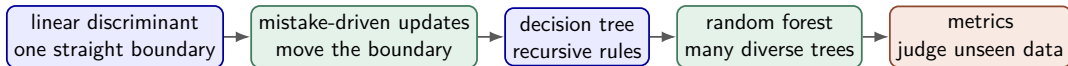
Week 16

Contents

1. Mission: Build, Improve, and Judge a Model
2. Linear Discriminants: Start with One Boundary
3. Decision Trees: Build a Boundary from Questions
4. Random Forests: Combine Many Diverse Trees
5. Classification Evaluation: Make the Decision Consequences Clear
6. Linear Regression and Regression Evaluation
7. Integration and Wrap-Up

**Mission: Build, Improve, and Judge a
Model**

Today's Story: From One Line to a Forest



Central question

How do we increase a model's flexibility without fooling ourselves about whether it actually improved?

Learning Objectives

By the end of class, students can:

1. interpret a linear discriminant score and its decision boundary;
2. update a linear boundary after a classification mistake;
3. explain how a decision tree selects and repeats feature splits;
4. explain why a random forest is usually more stable than one tree;
5. calculate and choose classification metrics from a confusion matrix;
6. fit a simple linear-regression model and interpret residuals; and
7. calculate MAE, MSE, RMSE, and R^2 for a regression task.

120-Minute Flow

1. **0–25 min:** linear scores, boundaries, and a manual prediction challenge.
2. **25–42 min:** perceptron updates and the limit of one straight line.
3. **42–65 min:** decision trees, impurity, recursive splitting, and overfitting.
4. **65–70 min:** break.
5. **70–86 min:** random forests and why they are useful for tabular production data.
6. **86–103 min:** classification evaluation, thresholds, and imbalanced data.
7. **103–116 min:** linear regression and regression evaluation.
8. **116–120 min:** exit ticket and homework briefing.

Classification and Regression Ask Different Questions

Classification

Predict a category.

$$y \in \{0, 1\}$$

Example: Will a transformer need inspection this week?

The model and evaluation metric must match the type of target.

Regression

Predict a number.

$$y \in \mathbb{R}$$

Example: What will tomorrow's peak load be in MW?

Linear Discriminants: Start with One Boundary

A Two-Feature Inspection Dataset

For each transformer, use two standardized features:

$$x_1 = \text{recent load stress}, \quad x_2 = \text{temperature stress}.$$

The label is

$$y = 1 \text{ inspect soon}, \quad y = 0 \text{ routine monitoring}.$$

Why standardize?

A feature measured in thousands should not dominate another feature merely because its numerical units are larger.

The Simplest Discriminant: A Weighted Score

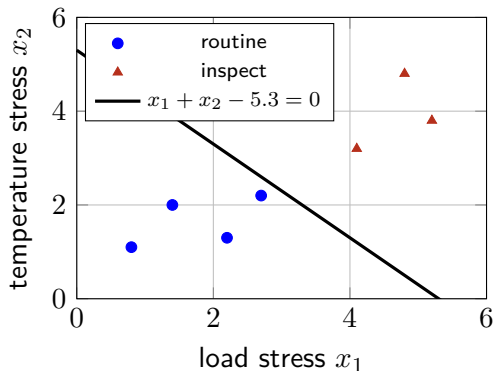
Linear discriminant function

$$z = w_1x_1 + w_2x_2 + b = w^T x + b.$$

$$\hat{y} = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0. \end{cases}$$

- A positive weight means larger feature values push toward class 1.
- A negative weight pushes toward class 0.
- The bias moves the boundary without changing its angle.

The Equation $w^T x + b = 0$ Draws the Boundary



Geometry

w points toward increasing score and is perpendicular to the line.

Distance intuition

A large positive z means the point is confidently on the class-1 side only when the feature scale and model are meaningful.

Manual Prediction Challenge

Use

$$z = 0.8x_1 + 0.6x_2 - 3.0, \text{quad}\hat{y} = \mathbf{1}[z \geq 0].$$

Transformer	x_1	x_2	Your z and prediction
A	1.0	1.5	
B	3.0	1.0	
C	2.5	2.5	
D	4.0	3.0	

Pair question

Which feature has the stronger effect per standardized unit? How would increasing b change the number of inspection predictions?

How Can We Choose the Boundary?

Several classic linear methods use the same form $w^T x + b$ but learn it differently.

Method	Main idea
Linear discriminant analysis	Use class centers and a shared estimate of within-class spread.
Logistic regression	Choose weights that make observed labels probable.
Linear support-vector machine	Seek a separating boundary with a large margin.
Perceptron	Move the boundary whenever an example is misclassified.

Today we use the perceptron because every update can be calculated by hand.

Iterative Improvement: Learn from a Mistake

Encode labels as $y \in \{-1, +1\}$. A point is correct when

$$y(w^T x + b) > 0.$$

If it is wrong or exactly on the boundary, update

$$w \leftarrow w + \eta y x, b \leftarrow b + \eta y,$$

where $\eta > 0$ is the learning rate.

Interpretation

A missed positive pulls the boundary toward classifying that point positive. A false positive pushes it in the opposite direction.

One Perceptron Update by Hand

Start with

$$w = [0.5, -0.2]^T, \quad b = -0.1, \quad \eta = 0.1.$$

A training example has $x = [1, 2]^T$ and $y = +1$.

1. Score: $z = 0.5(1) - 0.2(2) - 0.1 = 0$.
2. Because $yz = 0$, update the parameters.
3. $w_{\text{new}} = [0.5, -0.2]^T + 0.1(+1)[1, 2]^T = [0.6, 0]^T$.
4. $b_{\text{new}} = -0.1 + 0.1 = 0$.
5. New score for the same point: $z_{\text{new}} = 0.6 > 0$.

One example improved; unseen-data performance is still unknown.

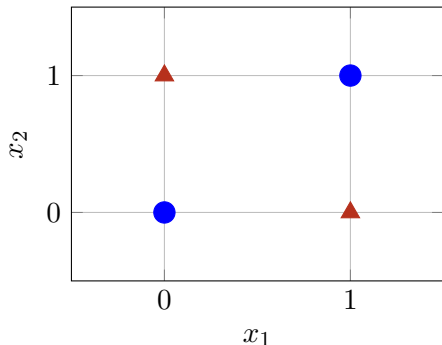
The Mistake-Driven Training Loop

```
1 for epoch in range(num_epochs):
2     mistakes = 0
3     for x_i, y_i in zip(X, y):    # y_i is -1 or +1
4         score = w @ x_i + b
5         if y_i * score <= 0:
6             w = w + learning_rate * y_i * x_i
7             b = b + learning_rate * y_i
8             mistakes += 1
9     print(epoch, mistakes)
```

Important limit

If the classes cannot be separated by one straight boundary, the perceptron may continue making updates without finding a perfect separator.

One Straight Line Cannot Solve Every Pattern



XOR pattern

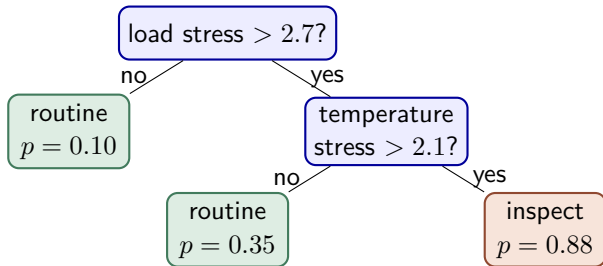
No single line separates the blue points from the red points.

Options include:

- create nonlinear features;
- use a neural network; or
- divide the space into multiple regions with a tree.

Decision Trees: Build a Boundary from Questions

A Decision Tree Is a Sequence of Questions



Each internal node asks one question. Each leaf stores a prediction based on the training examples that reached it.

Tree Anatomy

Term	Meaning
Root	The first question, containing all training examples.
Split	A feature and threshold that divide examples into child nodes.
Branch	The answer path, such as yes/no or left/right.
Leaf	A final prediction or class probability.
Depth	Number of questions on the longest root-to-leaf path.

Training is recursive

Choose a useful split, divide the data, and repeat the process inside each child node.

A Good Split Creates Purer Groups

For binary classification, Gini impurity is

$$G = 1 - p_0^2 - p_1^2.$$

Node contents	p_0	p_1	Gini
5 routine, 5 inspect	.50	.50	.50
8 routine, 2 inspect	.80	.20	.32
10 routine, 0 inspect	1.00	0	0

Lower impurity means the node is closer to containing one class only.

Compare Candidate Splits with Weighted Impurity

A parent has 10 examples: 5 routine and 5 inspect, so $G_{\text{parent}} = .50$.

Candidate split A produces:

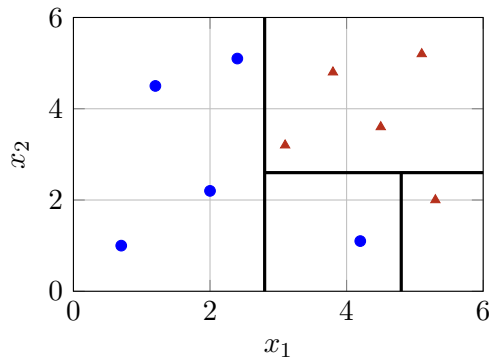
- left: 4 routine, 1 inspect, so $G_L = 1 - (4/5)^2 - (1/5)^2 = .32$;
- right: 1 routine, 4 inspect, so $G_R = .32$.

$$G_{\text{after}} = \frac{5}{10}(.32) + \frac{5}{10}(.32) = .32.$$

$$\text{impurity decrease} = .50 - .32 = .18.$$

The training algorithm compares many feature-threshold pairs and selects a large decrease.

Recursive Splits Create a Nonlinear Boundary



Key idea

Each threshold makes an axis-aligned cut. Repeated cuts form rectangles and a piecewise boundary.

More depth increases flexibility, but also increases the ability to memorize noise.

A Deep Tree Can Memorize the Training Set

	Shallow tree	Very deep tree
Training error	usually higher	can approach zero
Boundary	simple regions	many tiny regions
Sensitivity	lower	high: one point can alter the tree
Main risk	underfitting	overfitting

Controls

Tune `max_depth`, `min_samples_leaf`, and pruning using validation data or cross-validation—not the final test set.

Train and Inspect One Tree

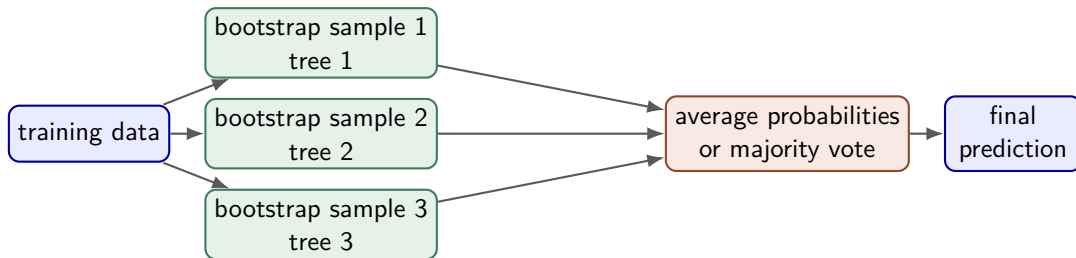
```
1 from sklearn.tree import DecisionTreeClassifier, plot_tree
2
3 tree = DecisionTreeClassifier(
4     max_depth=3,
5     min_samples_leaf=5,
6     class_weight="balanced",
7     random_state=16,
8 )
9 tree.fit(X_train, y_train)
10 probability = tree.predict_proba(X_test)[: , 1]
11 prediction = (probability >= 0.5).astype(int)
```

The test set is not a tuning set

Choose depth and other settings before the final test evaluation.

Random Forests: Combine Many Diverse Trees

One Tree Is Unstable; a Forest Averages



A random forest deliberately creates different trees, then reduces their individual instability by averaging.

Where Does the Diversity Come From?

Each tree receives two sources of randomness:

1. **Bootstrap examples:** sample training rows with replacement.
2. **Random feature subsets:** at each split, consider only some features.

Why not train identical trees?

Averaging identical errors changes nothing. Diversity makes it less likely that every tree follows the same noisy pattern.

Bias–variance intuition

A forest may keep the flexibility of deep trees while reducing variance, though it cannot repair biased data or missing predictive information.

Why Random Forests Are an Enduring Production Baseline

For many tabular datasets, random forests offer a practical combination:

- model nonlinear effects and feature interactions;
- require little feature scaling and limited preprocessing;
- work with mixed signal strengths and many irrelevant features;
- train trees in parallel and predict reliably with fixed code paths;
- provide class probabilities and several inspection tools; and
- establish a strong baseline before using a more complex model.

Not universally best

Forests can be large, cannot extrapolate regression trends well, and may lose to gradient boosting or specialized neural models. Measure on the actual task.

Single Tree versus Random Forest

Property	One decision tree	Random forest
Interpretability	easy to draw if shallow	harder; inspect summaries or individual trees
Stability	sensitive to small data changes	usually more stable
Accuracy	useful baseline	often stronger on tabular data
Speed/size	small and fast	more memory and prediction work
Overfitting control	depth and pruning	averaging plus tree controls

Train a Random Forest

```
1 from sklearn.ensemble import RandomForestClassifier
2
3 forest = RandomForestClassifier(
4     n_estimators=300,
5     max_depth=None,
6     min_samples_leaf=3,
7     max_features="sqrt",
8     class_weight="balanced",
9     n_jobs=-1,
10    random_state=16,
11 )
12 forest.fit(X_train, y_train)
13
14 p_test = forest.predict_proba(X_test)[: , 1]
15 y_test_pred = (p_test >= 0.5).astype(int)
```

Fixing `random_state` makes the experiment reproducible; it does not make the model non-random internally.

Feature Importance Is a Clue, Not a Cause

A forest can report which features helped its splits, but:

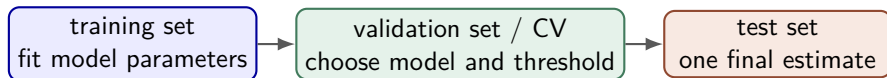
- importance does not prove that changing the feature changes the outcome;
- correlated features can divide credit unpredictably;
- impurity-based importance can favor variables with many possible split points; and
- importance says little about whether a feature raises or lowers one prediction.

Better question

Does performance on unseen data decrease when this feature's information is disrupted?
Permutation importance can help answer that predictive question.

Classification Evaluation: Make the Decision Consequences Clear

Separate Training, Tuning, and Final Evaluation



Data leakage

Any information from the test set that changes preprocessing, features, hyperparameters, or thresholds makes the final score too optimistic.

Every Binary Prediction Enters One Cell

		Predicted	
		Inspect	Routine
Actual	Inspect	TP	FN
	Routine	FP	TN

- **False negative:** a transformer needing inspection is missed.
- **False positive:** an unnecessary inspection is scheduled.

Before choosing a metric, state which error is more costly and why.

Four Core Classification Metrics

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN},$$

$$\text{Precision} = \frac{TP}{TP + FP},$$

$$\text{Recall} = \frac{TP}{TP + FN},$$

$$F_1 = 2 \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Accuracy How often is the label correct overall?

Precision Among inspection alerts, how many truly needed inspection?

Recall Among units needing inspection, how many did we catch?

F1 Are precision and recall jointly strong?

Worked Confusion-Matrix Example

Suppose $TP = 18$, $TN = 70$, $FP = 8$, and $FN = 4$.

$$\text{Accuracy} = (18 + 70)/100 = .88,$$

$$\text{Precision} = 18/(18 + 8) = .692,$$

$$\text{Recall} = 18/(18 + 4) = .818,$$

$$F_1 = 2(.692)(.818)/(.692 + .818) = .750.$$

Read the numbers in words

The model catches about 82% of inspection needs, and about 69% of its inspection alerts are correct.

Why Accuracy Can Hide Failure

Suppose only 10 of 1000 transformers need inspection. A model that always predicts routine has

$$\text{Accuracy} = 990/1000 = 99\%, \quad \text{Recall} = 0/10 = 0.$$

Imbalanced-data lesson

High accuracy can coexist with complete failure on the rare class. Compare against a simple baseline and report class-sensitive metrics.

Useful additions include precision–recall curves, per-class counts, and performance at an operational threshold.

The Threshold Is a Policy Choice

A model may output an inspection probability $\hat{p}$. We choose class 1 when $\hat{p} \geq t$.

Threshold change	Typical effect	Operational meaning
lower t	recall up, precision often down	inspect more units
higher t	precision often up, recall down	reserve inspections for strong alerts

Choose t on validation data using error costs, staffing capacity, and safety requirements. Do not assume 0.5 is automatically correct.

A Responsible Model Comparison Table

Model	Threshold	TP	FP	FN	Precision	Recall
majority baseline	—	0	0	22	—	0.00
linear discriminant	.45	14	7	8	.67	.64
decision tree	.50	16	10	6	.62	.73
random forest	.38	19	8	3	.70	.86

Do not select from one number alone

Also compare uncertainty across validation folds, latency, memory, calibration, and the real cost of each error type.

Linear Regression and Regression Evaluation

Linear Regression Predicts a Continuous Number

For one feature,

$$\hat{y} = \beta_0 + \beta_1 x.$$

For several features,

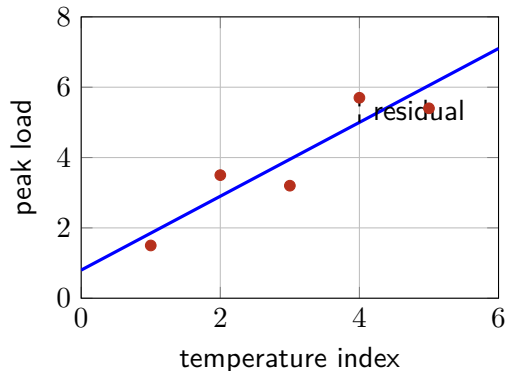
$$\hat{y} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p.$$

Peak-load example

Inputs may include temperature forecast, weekday indicator, yesterday's peak, and expected solar output. The target is tomorrow's peak load in MW.

The output is a number, not a class probability or thresholded decision.

Residuals Measure Vertical Prediction Errors



$$e_i = y_i - \hat{y}_i.$$

Sign

Positive residual: prediction was too low.

Negative residual: prediction was too high.

Ordinary Least Squares Chooses the Line

The most common fitting rule minimizes the sum of squared residuals:

$$\min_{\beta_0, \beta_1, \dots, \beta_p} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

- Squaring prevents positive and negative residuals from canceling.
- Large errors receive more weight than small errors.
- The coefficients describe associations conditional on the included features; they do not automatically prove causation.

Evaluate out of sample

A fitted line can look good on its training points and still predict future days poorly.

Core Regression Metrics

Let $e_i = y_i - \hat{y}_i$.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |e_i|,$$

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n e_i^2,$$

$$\text{RMSE} = \sqrt{\text{MSE}},$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

MAE and RMSE use the target's units; MSE uses squared units; R^2 compares against predicting the test-set mean.

Worked Regression-Metric Example

Suppose

$$y = [10, 12, 14, 16], \quad \hat{y} = [11, 11, 15, 13].$$

Residuals are $[-1, 1, -1, 3]$.

$$\text{MAE} = (1 + 1 + 1 + 3)/4 = 1.5,$$

$$\text{MSE} = (1 + 1 + 1 + 9)/4 = 3,$$

$$\text{RMSE} = \sqrt{3} \approx 1.73.$$

Interpretation

The model misses by 1.5 target units on average under absolute error; RMSE is larger because the error of 3 receives extra weight.

What Does R^2 Mean?

Value	Interpretation on the evaluated dataset
$R^2 = 1$	perfect predictions
$R^2 = 0$	same squared-error performance as predicting the mean
$R^2 < 0$	worse than the mean baseline

Common misconception

$R^2 = .80$ does not mean that every prediction is 80% accurate, and it does not reveal the error size in MW. Report MAE or RMSE too.

Choose the Regression Metric to Match the Cost

Metric	Prefer it when...
MAE	each unit of error has roughly equal operational cost and robustness to outliers matters.
RMSE	large misses are especially dangerous or expensive.
R^2	you want a scale-free comparison with a mean baseline on the same target and dataset.

Always include a baseline such as yesterday's load, a seasonal average, or the training-set mean.

A Small, Honest Regression Pipeline

```
1 from sklearn.linear_model import LinearRegression
2 from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
3
4 model = LinearRegression()
5 model.fit(X_train, y_train)
6 y_pred = model.predict(X_test)
7
8 mae = mean_absolute_error(y_test, y_pred)
9 mse = mean_squared_error(y_test, y_pred)
10 rmse = mse ** 0.5
11 r2 = r2_score(y_test, y_pred)
```

For time-dependent grid data, split chronologically when the goal is to predict the future; random splitting can leak future patterns into training.

Integration and Wrap-Up

Class Activity: Model Card for an Inspection System

In teams, compare a linear classifier, one tree, and a random forest. Record:

1. target and positive-class definition;
2. train/validation/test split strategy;
3. selected threshold and operational reason;
4. TP, TN, FP, FN, precision, recall, and F1;
5. training time, prediction time, and model size;
6. one likely data bias or failure mode; and
7. which model you would deploy and what monitoring it requires.

A production decision is more than a leaderboard score

Reliability, cost, latency, interpretability, maintainability, and changing data all matter.

Common Misconceptions

- **“More flexible always means better.”** Flexibility can fit signal or memorize noise.
- **“A tree learns like a neural network.”** A standard tree greedily selects discrete splits; it does not use backpropagation.
- **“Random forest means random predictions.”** Randomness occurs during construction; a fixed trained model gives repeatable predictions.
- **“Accuracy is enough.”** Rare but important classes can be completely missed.
- **“High feature importance proves causation.”** It reports predictive use inside a model, not an intervention effect.
- **“High R^2 means small errors.”** Error magnitude still requires MAE or RMSE.

Exit Ticket

Answer each in one sentence or calculation:

1. For $z = 2x_1 - x_2 - 1$, classify the point $(1, 2)$ using threshold $z \geq 0$.
2. What does a perceptron do after a correctly classified example?
3. Why can a tree represent a boundary that one line cannot?
4. Name the two sources of randomness in a random forest.
5. If $TP = 12$, $FP = 3$, and $FN = 8$, calculate precision and recall.
6. Which regression metric penalizes a single large error more: MAE or RMSE?
7. What does a negative test-set R^2 imply?

Homework for This Week

Manual classification

Calculate linear scores, perform perceptron updates, build confusion matrices, and explain threshold tradeoffs.

Model comparison

Train a linear classifier, decision tree, and random forest on the same split; compare both predictive and operational properties.

Regression extension

Fit a linear regression, manually verify $\text{MAE}/\text{MSE}/\text{RMSE}/R^2$, and diagnose the largest residuals.

Key Takeaways

1. A linear discriminant separates classes with the sign of $w^T x + b$.
2. Mistake-driven updates can move a linear boundary, but one line cannot express every pattern.
3. Trees build nonlinear boundaries by recursively splitting the feature space.
4. Random forests average diverse trees to improve stability and form a strong tabular baseline.
5. Classification metrics describe different consequences; the threshold is an operational decision.
6. Regression predicts numbers and should be evaluated with error metrics plus a meaningful baseline.
7. Honest unseen-data evaluation matters more than training-set perfection.